

LEHRSTUHL FÜR RECHNERARCHITEKTUR UND PARALLELE SYSTEME

Aspekte der systemnahen Programmierung bei der Spieleentwicklung

Burning Ship-Fraktal (A217)

Projektaufgabe – Aufgabenbereich Bildverarbeitung

1 Organisatorisches

Auf den folgenden Seiten finden Sie die Aufgabenstellung zu Ihrer Projektaufgabe für das Praktikum. Die Rahmenbedingungen für die Bearbeitung werden in der Praktikumsordnung festgesetzt, die Sie über die Praktikumshomepage¹ aufrufen können.

Wie in der Praktikumsordnung beschrieben, sind die Aufgaben relativ offen gestellt. Besprechen Sie diese innerhalb Ihrer Gruppe und konkretisieren Sie die Aufgabenstellung. Die Teile der Aufgabe, in denen Assembler-Code anzufertigen ist, sind für die x86-64 Architektur unter Verwendung der SSE-Erweiterungen (bis SSE4.2) zu schreiben; alle anderen Bestandteile der Hauptimplementierung sind in C nach dem C17-Standard anzufertigen.

Der **Abgabetermin** ist **Sonntag 1. Februar 2026, 23:59 Uhr**. Die Abgabe erfolgt per Git in das für Ihre Gruppe eingerichtete Projektrepository auf dem main Branch. Bitte beachten Sie die in der README.md angegebene Liste von abzugebenden Dateien.

Die **Abschlusspräsentationen** finden in der Zeit vom **9.03.2026 – 13.03.2026** statt. Weitere Informationen werden noch bekannt gegeben. Beachten Sie, dass die Folien für die Präsentation am obigen Abgabetermin im PDF-Format abzugeben sind und keine nachträglichen Änderungen akzeptiert werden können.

Bei Fragen/Unklarheiten in Bezug auf den Ablauf und die Aufgabenstellung wenden Sie sich bitte an Ihre:n Tutor:in.

Wir wünschen Ihnen viel Erfolg und Freude bei der Bearbeitung Ihrer Aufgabe!

Mit freundlichen Grüßen
Die Praktikumsleitung

PS: Vergessen Sie nicht, sich rechtzeitig in TUMonline zur Prüfung anzumelden. Dies ist Voraussetzung für eine erfolgreiche Teilnahme am Praktikum im laufenden Semester.

¹<https://asp.caps.in.tum.de>

2 Burning Ship-Fraktal

2.1 Überblick

Im Zuge Ihrer Projektaufgabe werden Sie theoretisches Wissen aus der Mathematik im Anwendungszusammenhang verwenden, um einen Algorithmus in Assembler zu implementieren. Sie konzentrieren sich dabei auf das Feld des *Image Processing*, in welchem Pixelbilder, wie sie typischerweise Digitalkameras produzieren, als Eingabe für bestimmte Algorithmen verwendet werden und mathematische Überlegungen dadurch sichtbar gemacht werden.

2.2 Funktionsweise

In dieser Aufgabe befassen Sie sich mit dem Burning Ship-Fraktal. Dieses ist gegeben durch iterative Anwendung der komplexen Gleichung

$$z_{k+1} = (|\operatorname{Re}(z_k)| + |\operatorname{Im}(z_k)|i)^2 + c \quad (k \geq 0)$$

und Betrachtung des Wertes z_k innerhalb der komplexen Ebene. Dabei wählt man $z_0 = 0$ und Real- sowie Imaginärteil von c gemäß der Koordinaten in der komplexen Ebene. Durch Variation von c und Betrachtung von z_n für große n lässt sich so das charakteristische Burning Ship-Fraktal in der komplexen Ebene darstellen.

2.3 Aufgabenstellungen

Ihre Aufgaben lassen sich in die Bereiche Konzeption (theoretisch) und Implementierung (praktisch) aufteilen. Sie können (müssen aber nicht) dies bei der Verteilung der Aufgaben innerhalb Ihrer Arbeitsgruppe ausnutzen. Antworten auf konzeptionelle Fragen sollten an den passenden Stellen in Ihrem Vortrag in angemessenem Umfang erscheinen. Entscheiden Sie nach eigenem Ermessen, ob Sie im Rahmen Ihres Abschlussvortrags auch auf konzeptionelle Fragen eingehen. Die Antworten auf die Implementierungsaufgaben werden durch Ihren Code reflektiert.

2.3.1 Theoretischer Teil

- Berechnen Sie Real- sowie Imaginärteil von z_{k+1} in Abhängigkeit von Real- sowie Imaginärteil von z_k explizit. Dies ist die Iterationsvorschrift, die Ihr Algorithmus verwenden soll.
 - Sofern z_k beschränkt ist, färben Sie den Pixel, der zu Real- sowie Imaginärteil von c korrespondiert schwarz, da er zum Burning Ship gehört. Wie verfahren Sie in dieser Hinsicht mit Pixeln, die zu Koordinaten korrespondieren, für die z_k divergiert?
-

- Der Algorithmus lässt sich leicht parallelisieren. Entwerfen Sie einen parallelen Algorithmus, der die SIMD-Einheit Ihrer Ziel-Architektur möglichst optimal verwendet.

2.3.2 Praktischer Teil

- Implementieren Sie in der Datei mit Ihrem Assemblercode die Funktion:

```
void burning_ship(float complex start, size_t width, size_t height,  
                 float res, unsigned n, unsigned char* img)
```

Die Funktion bekommt den rechteckigen Ausschnitt des Burning Ship-Fraktal in der komplexen Ebene, der durch `start`, `width`, `height` und die gewünschte Auflösung (Schrittweite pro Bildpixel) `res` gekennzeichnet wird, übergeben. Als weitere Parameter erhält die Funktion die maximale Anzahl der Iterationen `n` sowie einen Zeiger auf einen Speicherbereich, der groß genug ist, um die berechneten Bitmap-Daten des Ergebnisses zu speichern. Allokieren Sie hierzu in Ihrem Rahmenprogramm einen Buffer passender Größe. Legen Sie Ihrer Ausgabe eine sinnvolle Farbpalette zugrunde.

- Wie Sie wissen, dürfen Sie alle I/O-Operationen im Rahmenprogramm in C durchführen. Wir schreiben Ihnen kein Ausgabeformat vor, legen Ihnen aber nahe, mit unkomprimierten, 24-bit BMP-Dateien zu arbeiten. Das Schreiben des Headers und der Daten kann daher in C erfolgen.

2.3.3 Rahmenprogramm

Ihr Rahmenprogramm muss bei einem Aufruf die folgenden Optionen entgegennehmen und verarbeiten können. Wenn möglich soll das Programm sinnvolle Standardwerte definieren, sodass nicht immer alle Optionen gesetzt werden müssen. Wird eine Option mit Argument gesetzt, so ist das entsprechende Argument zu benutzen. Die Reihenfolge der Optionen bei einem gültigen Aufruf muss irrelevant sein. Wir empfehlen Ihnen, die Kommandozeilenparameter mittels `getopt_long`² zu parsen.

- `-V <Zahl>` — Die Implementierung, die verwendet werden soll. Hierbei soll mit `-V 0` Ihre Hauptimplementierung verwendet werden. Wenn diese Option nicht gesetzt wird, soll ebenfalls die Hauptimplementierung ausgeführt werden.
- `-B<Zahl>` — Falls gesetzt, wird die Laufzeit der angegebenen Implementierung gemessen und ausgegeben. Das *optionale* Argument dieser Option gibt die Anzahl an Wiederholungen des Funktionsaufrufs an, z. B. `-B` oder `-B5`.
- `-s <Realteil>, <Imaginärteil>` — Startpunkt der Berechnung
- `-d <Zahl>, <Zahl>` — Breite und Höhe des Bildausschnitts

²Siehe man 3 `getopt_long` für Details

- `-n <Zahl>` — Maximale Anzahl an Iterationen pro Bildpixel
- `-r <Floating Point Zahl>` — Schrittweite pro Bildpixel
- `-o <Dateiname>` — Ausgabedatei
- `-h|--help` — Eine Beschreibung aller Optionen des Programms und Verwendungsbeispiele werden ausgegeben und das Programm danach beendet. Die Beschreibung soll mit "Help Message" anfangen.

Sie dürfen weitere Optionen implementieren, beispielsweise um vordefinierte Testfälle zu verwenden. Ihr Programm muss jedoch nur unter Verwendung der oben genannten Optionen verwendbar sein. Beachten Sie ebenfalls, dass Ihr Rahmenprogramm etwaige Randfälle korrekt abfangen muss und im Falle eines Fehlers mit einer Fehlermeldung, die auf den konkreten Fehler hinweist und einer kurzen Erläuterung zur Benutzung **terminieren** sollte. Fehlermeldungen jeglicher Art sollen auf `stderr` ausgegeben werden. Die Wertebereiche der Argumente dürfen nicht unnötig eingeschränkt werden.

2.4 Allgemeine Bewertungshinweise

Beachten Sie grundsätzlich alle in der Praktikumsordnung angegebenen Hinweise. Die folgende Liste konkretisiert einige der Bewertungspunkte:

- Diese PDF-Datei darf nicht verändert, gelöscht oder umbenannt werden.
- Die in `README.md` geforderte Struktur des Projektrepositorys muss eingehalten werden.
- Der Exit-Code Ihrer Implementierung muss die erfolgreiche Ausführung oder das Auftreten eines Fehlers widerspiegeln. Benutzen Sie dafür die Makros aus `stdlib.h`.
- Stellen Sie unbedingt sicher, dass Ihre Implementierung auf der Referenzplattform des Praktikums (`1xhalle`) kompiliert und vollständig funktionsfähig ist.
- Die Implementierung soll mit GCC/GNU as kompilieren. Verwenden Sie keinen Inline-Assembler. Achten Sie darauf, dass Ihr Programm keine x87-FPU- oder MMX-Instruktionen und SSE-Erweiterungen nur bis SSE4.2 verwendet. Andere ISA-Erweiterungen (z.B. AVX, BMI1) dürfen Sie nur benutzen, sofern Ihre Implementierung auch auf Prozessoren ohne derartige Erweiterungen lauffähig ist.
- Sie dürfen die angegebenen Funktionssignaturen, bis auf die Änderung des Funktionsnamens für die Benennung der unterschiedlichen Implementierungen (siehe nächster Punkt) *nicht* ändern
- Verwenden Sie den angegebenen Funktionsnamen bzw. die angegebenen Funktionsnamen für Ihre Hauptimplementierung bzw. Hauptimplementierungen. Für alle weiteren Implementierungen gilt folgendes Schema: Hinter den Funktionsnamen wird das Suffix „_V1“, „_V2“, usw. angehängt.

- I/O-Operationen dürfen grundsätzlich in C implementiert werden.
 - Denken Sie daran, das Laufzeitverhalten Ihres Codes zu testen (Sichere Programmierung, Korrektheit/Genauigkeit, Performanz) und behandeln Sie *alle möglichen Eingaben*, auch Randfälle. Ziehen Sie ggf. alternative Implementierungen als Vergleich heran.
 - Eingabedateien, welche Sie generieren, um Ihre Implementierungen zu testen, sollten mit abgegeben werden; größere Eingaben sollten stattdessen stark komprimiert oder (bevorzugt) über ein abgegebenes Skript generierbar sein.
 - Stellen Sie Performanz-Ergebnisse nach Möglichkeit grafisch dar.
 - Vermeiden Sie unscharfe Grafiken und Screenshots von Code.
 - Geben Sie die Folien für Ihre Abschlusspräsentation im PDF-Format ab. Achten Sie auf hinreichenden Kontrast (schwarzer Text auf weißem Grund!) und eine angemessene Schriftgröße. Verwenden Sie 16:9 als Folien-Format und fügen Sie Foliennummern hinzu.
-